

Makemore

Part 5 - Building a WaveNet

From the flat MLP to a hierarchical, tree-like model that crosses 2.0

A comprehensive, cell-by-cell study handbook for the fifth video in Andrej Karpathy's *makemore* series — “Building makemore Part 5: Building a WaveNet.” This expanded edition explains the **what**, the **why**, and the **how** behind every module, every shape transformation, and every architectural choice as we evolve our two-layer MLP into a three-stage hierarchical model inspired by DeepMind's WaveNet.

Compiled by: **Shreyas Pradeepkumar Khandale** · GitHub: **sherurox**

Companion to: `makemore_part5_cnn1.ipynb`

Edition: *Expanded - June 2026*

Where this handbook sits in the series

Parts 1 through 4 stayed inside one architecture: a two-layer MLP that took a small fixed window of previous characters, embedded each into a learned vector, concatenated the embeddings, and pushed the resulting flat vector through a single hidden layer to predict the next character. Part 2 built it. Part 3 fixed its initialization and added Batch Normalization. Part 4 opened up the autograd black box and derived every gradient by hand. Throughout all of that, the architecture itself did not change.

Part 5 is the lecture where the architecture finally changes. We outgrow the flat MLP for a specific reason — squashing eight characters of context into a single hidden layer wastes the depth we have available — and replace it with a hierarchical, tree-like model in which every level of depth fuses information from a wider window of context. The structure is borrowed from DeepMind's WaveNet (2016), originally designed for autoregressive audio generation. Along the way we also wrap the rest of our code into PyTorch-style modules (`Embedding`, `Flatten`, `Sequential`), discover a quietly important fact about matrix multiplication broadcasting, find and fix a Batch Normalization bug, and finally cross the 2.0 validation-loss barrier that has stood since Part 2.

What is actually new compared to Part 3

- **PyTorch-ified module library.** Three new layer classes — `Embedding`, `FlattenConsecutive`, and `Sequential` — complete the small framework we have been building, so the whole network is now expressed as a single `Sequential(...)` call.

- **Block size increases from 3 to 8.** The context window triples without any other architectural change, and this alone moves the validation loss from 2.10 to 2.027.
- **The hierarchical (tree-like) WaveNet topology.** Three stages of `FlattenConsecutive(2)` -> `Linear` -> `BatchNorm` -> `Tanh`, each fusing pairs of adjacent embeddings, replace the single flat hidden layer.
- **The matmul broadcasting insight.** A linear layer's `x @ W` only operates on the last dimension. Leading dimensions are treated as batch. This is the technical foundation that makes the hierarchical model work efficiently.
- **A BatchNorm bug with 3D inputs.** Our Part 3 `BatchNorm` reduced over axis 0 only, which is wrong when the input has shape (batch, time, channels). We diagnose the bug by inspecting `running_mean.shape` and fix it by reducing over both leading axes.
- **Crossing 2.0.** After scaling up to `n_embd=24`, `n_hidden=128` (76K parameters), the validation loss reaches **1.993** — the first time the series has crossed below 2.0.
- **Convolution preview.** A hint at what the next stage looks like: the same hierarchical computation can be slid efficiently across the input sequence using convolutions, which would let us share work across overlapping windows.

Preface: How to use this handbook

This handbook is the fifth in a coordinated series covering Andrej Karpathy's *makemore* lectures. It is organized as a guided walk through the `makemore_part5_cnn1.ipynb` notebook, with each chapter mapping to a coherent group of cells. The format is identical to the Parts 1–4 expanded editions so the five documents can be read side-by-side as one continuous reference.

Reading mode 1: Linear study

Read the handbook end-to-end while keeping the notebook open in a separate window. Each chapter introduces the concept, presents the relevant code cell exactly as it appears in the notebook, and then explains what the code is doing, why it is structured that way, and how it fits into the larger story. Part 5 is unique in the series in that it spends much of its energy on what Karpathy calls “PyTorch-ifying” the code — building up the small layer-library framework that mirrors `torch.nn`. Reading through this material once carefully will leave you with a much firmer grasp of what PyTorch's actual modules do under the hood.

Reading mode 2: Reference lookup

Once you have completed a first pass, the handbook also functions as a reference. The Table of Contents lists every concept with its chapter number, and the Glossary at the end provides one-line definitions for every technical term used in the lecture. The diagnostic cards in Chapter 27 capture the most common bugs you will hit when working with multi-dimensional BatchNorm, shape mismatches, and the train/eval distinction. They are short, scannable, and worth keeping handy whenever you find yourself debugging a network whose shapes do not line up.

Conventions used in this handbook

The handbook uses a small set of visual conventions consistently throughout the document. They are identical to the conventions in Parts 1–4.

- **Code blocks** appear in a fixed-width font on a warm beige background. Code is reproduced verbatim from the notebook so you can match each block to the cell that produced it.
- **Cell labels** precede every code block in small blue type (for example, “*Cell — The hierarchical model.*”).
- **Callout boxes** (yellow background, rust-coloured title) highlight Karpathy's spoken framing, important warnings, or moments of insight.
- **Info boxes** (light blue background) contain conceptual definitions and theoretical background. They are useful for first-time learners and can be skimmed by readers already familiar with the material.
- **Part dividers** separate the five major sections of the handbook with a full-width banner.
- **Diagnostic cards** (Chapter 27) present each common pathology in its own clearly bounded card with three labeled rows: symptom, diagnosis, and action.

Prerequisites

This handbook assumes you have completed the Part 1–3 handbooks (or are comfortable with the MLP architecture, the embedding layer, `F.cross_entropy`, `BatchNorm`, and the train/dev/test split). Part 4 is helpful for understanding the gradient-level intuition behind why certain architectural choices matter, but it is not strictly required for Part 5. Mathematical prerequisites are light: you need to be comfortable with tensor shapes, broadcasting, and the basic syntax of PyTorch indexing and reshaping.

How to extract maximum value from this material

Part 5 is the lecture in which the makemore series finally graduates from a flat MLP into a deeper, structured architecture. The single most valuable thing you can take away is the intuition for *why* hierarchical structure helps — not just “it adds layers” but specifically that each layer of depth fuses information from a wider window of context, preserving more of the input structure than a single squash would. The mechanics of how the `FlattenConsecutive` module rearranges shapes to make this work, and the `matmul` broadcasting fact that lets the same `Linear` layer operate on different positions in parallel, are the technical foundation. Once you have those, you understand the WaveNet architecture, and you are most of the way to understanding why convolutional networks are structured the way they are.

Contents

Front matter

Preface: How to use this handbook

PART I — Where Part 5 begins

1. The motivation: from MLP to a deeper, hierarchical model
2. The WaveNet paper: autoregressive prediction with progressive fusion
3. The starter code: returning to Part 3

PART II — PyTorch-ifying the code

4. Fixing the loss plot — the view-and-mean trick
5. The Embedding module
6. The Flatten module (initial naive version)
7. The Sequential container
8. The model.layers refactor
9. A bug from forgetting eval mode

PART III — Scaling context and the matmul insight

10. Increasing block size from 3 to 8
11. The baseline result of lazily scaling context
12. Why crushing information too fast is suboptimal
13. The matmul broadcasting insight
14. The FlattenConsecutive generalization

PART IV — The hierarchical architecture and BatchNorm bug

15. The squeeze trick for the final flatten
16. Stacking three FlattenConsecutive + Linear + BatchNorm + Tanh
17. Inspecting shapes through the network
18. Hierarchical with same parameter count: no improvement yet
19. The BatchNorm 3D bug: reducing over the wrong dimension
20. PyTorch's BatchNorm1d convention ($N \times C \times L$ vs $N \times L \times C$)
21. Scaling up: `n_embd` 24, `n_hidden` 128, crossing 2.0

PART V — Convolutions, takeaways, and reference

22. The convolution-is-a-for-loop insight
23. Variable reuse and receptive field overlap
24. What WaveNet's full architecture adds
25. The development process: docs, shapes, Jupyter prototyping

- 26. The full performance log across the series
- 27. Diagnostic cards: shape and BatchNorm pitfalls
- 28. Summary and where the series goes next
- 29. Series-level recap: where we have been since Part 1
- 30. Glossary

PART I

Where Part 5 begins

Why we are leaving the flat MLP behind, what the WaveNet paper actually proposes, and where the starter code picks up.

1. The motivation: from MLP to a deeper, hierarchical model

Chapter overview

Karpathy opens the lecture from a hotel room in Kyoto with a direct framing: the multi-layer perceptron we have been polishing for four lectures is structurally limited, and we are going to fix it. The limitation is not that the network is too small. It is that it takes three characters of context and immediately crushes them into a single hidden layer, throwing away all the temporal structure of the input in one step. This chapter explains why that is a problem and what we are going to do about it.

Imagine the network seeing the string ' . . d ' and trying to predict the next character. The MLP we built in Part 2 takes the three input character embeddings, concatenates them into a single 30-dimensional vector, and feeds them into a single hidden layer with 200 neurons. Every neuron in that hidden layer sees all three characters at once. There is no notion that the third character is more recent than the first, or that fusing ' . . ' first and then folding in ' d ' might preserve useful structure. The network has to learn all of that from scratch through training, with no architectural prior to guide it.

Why depth without structure is not enough

You might think: just add more layers. Make the MLP four layers deep instead of two. The problem is that no matter how many layers you stack, the very first layer still takes all three characters and crushes them into one hidden representation. The damage is done in the first multiplication. Adding depth after that point does not recover information that was lost at the beginning. What we want instead is depth that does meaningful work at every level — fusing a little bit of context at each step, rather than fusing everything at once.

The hierarchical alternative

The model we are going to build does the fusion gradually. We start with eight character embeddings (the new block size). At the first hidden layer, we fuse pairs of adjacent embeddings into bigram representations — four of them, one for each pair. At the second hidden layer, we fuse pairs of bigrams into four-gram representations — two of them. At the third hidden layer, we fuse the two four-grams into one eight-gram representation, which is the input to the final classifier. The structure is a binary tree: leaves are individual characters, the root is the full eight-character context, and every internal node fuses two children into a single richer representation.

Karpathy's framing

“We don't just want to feed them all into a single hidden layer because that squashes too much information too quickly. Instead we would like to make a deeper model that progressively fuses this information to make its guess about the next character in a sequence. As we make this architecture more complex we're actually going to arrive at something that looks very much like a WaveNet.”

2. The WaveNet paper: autoregressive prediction with progressive fusion

Chapter overview

WaveNet is a 2016 paper from DeepMind that introduced an autoregressive model for raw audio generation. The audio domain is different from ours — the model predicts raw waveform samples instead of characters — but the core modelling problem is identical: given a sequence of past tokens, predict the next one. The architecture has been influential beyond audio because the tree-like fusion structure it introduced applies to any sequence task. This chapter explains the structural idea we are borrowing and what we are deliberately leaving out.

What we are borrowing

The single idea we take from WaveNet is the hierarchical, tree-like topology: a stack of layers in which each layer fuses pairs of adjacent positions from the layer below. After one such layer, every output position has seen two input positions. After two such layers, every output position has seen four input positions. After three, eight. The number of positions an output can see grows exponentially with depth, while the per-layer fusion stays simple.

Why the exponential growth matters

A flat MLP needs $O(\text{context} * \text{hidden})$ parameters in the first layer alone to handle a context of a given length. A hierarchical model with binary fusion uses $O(\text{hidden}^2 * \log(\text{context}))$ parameters across all its layers. The savings get dramatic as the context grows. For our 8-character context the difference is modest, but for a 100-character context or longer, hierarchical structure is essentially the only practical choice.

What we are deliberately leaving out

WaveNet's full architecture has several components beyond the tree-like fusion that we will not implement in this lecture. Karpathy mentions them by name and defers them to future work:

- **Dilated causal convolutions.** The paper implements the tree topology using a specific kind of convolution that skips over input positions in a controlled way. We will implement the tree directly with our existing Linear layers and a clever FlattenConsecutive reshape. Same topology, different mechanics.
- **Gated linear units.** WaveNet uses a gated activation function instead of plain tanh, where the output of one linear projection is multiplied by the sigmoid of another. We stick with tanh.
- **Residual connections.** Each layer adds its input to its output, so gradients can flow more easily through deep stacks. We do not need this for our shallow three-stage network.
- **Skip connections.** Outputs from intermediate layers are added together and contribute directly to the final prediction. Also skipped in our implementation.

What is left after these omissions is just the tree-like topology, which is the single structural idea that most distinguishes WaveNet from a flat MLP. The other components are valuable but secondary — they make WaveNet train more reliably at large depths, but they do not change what the model is fundamentally doing.

3. The starter code: returning to Part 3

Chapter overview

Karpathy explicitly notes that Part 4 (manual backprop) was an aside. The starter code for Part 5 is not the Part 4 notebook; it is the Part 3 notebook, with very small cosmetic changes. This chapter walks through what is identical to Part 3 (most of it) and what is new (the layer module library, slightly expanded). If you have done Part 3 you can skim this chapter.

Cell - Imports and data loading (unchanged from Part 3)

```
import torch
import torch.nn.functional as F
import matplotlib.pyplot as plt

words = open('names.txt', 'r').read().splitlines()
chars = sorted(list(set(''.join(words))))
stoi = {s: i+1 for i, s in enumerate(chars)}
stoi['.'] = 0
itos = {i: s for s, i in stoi.items()}
vocab_size = len(itos) # 27

import random
random.seed(42)
random.shuffle(words)
```

Identical to Part 3. 27-character vocabulary including the special '.' token. Words shuffled with seed 42 so we get the same train/dev/test split as before.

Cell - Build the dataset (block_size = 8, not 3)

```
block_size = 8 # context length: how many characters to take in
```

```
def build_dataset(words):
    X, Y = [], []
    for w in words:
        context = [0] * block_size
        for ch in w + '.':
            ix = stoi[ch]
            X.append(context)
            Y.append(ix)
            context = context[1:] + [ix]
    X = torch.tensor(X)
    Y = torch.tensor(Y)
    print(X.shape, Y.shape)
    return X, Y
```

```
n1 = int(0.8 * len(words))
```

```

n2 = int(0.9 * len(words))
Xtr, Ytr = build_dataset(words[:n1])      # 80%
Xdev, Ydev = build_dataset(words[n1:n2])  # 10%
Xte, Yte = build_dataset(words[n2:])      # 10%

```

The only change here is `block_size = 8` instead of 3. The same sliding-window construction creates training examples; each example now has eight characters of context rather than three. The training set is the same words, but the number of (context, target) pairs is slightly different because we pad each word with eight zeros at the start instead of three. Karpathy briefly increases the block size for the dataset-building step here even though we will not actually use the longer context until Chapter 10.

How the dataset grows with `block_size`

Each name produces roughly `len(name) + 1` training examples, with the first ones having mostly zero-padding and the later ones having real characters. Increasing the block size does not multiply the number of examples; it just makes each example have a longer context window. The total number of examples stays around 228K for the full training set.

The Part 3 layer modules, lightly refined

Karpathy then copy-pastes the layer classes from Part 3 (`Linear`, `BatchNorm1d`, `Tanh`) into Part 5 with minor changes. One change worth flagging: the random number generator `g` that Part 3 passed explicitly into every layer is now a global RNG, seeded once at the top with `torch.manual_seed(42)`. This is purely cosmetic but cleaner. The next chapter (and the following ones) add three new modules to this set.

Cell - The Part 3 layer modules, ready to be extended

```

class Linear:
    def __init__(self, fan_in, fan_out, bias=True):
        self.weight = torch.randn((fan_in, fan_out)) / fan_in ** 0.5  # Kaiming init
        self.bias = torch.zeros(fan_out) if bias else None
    def __call__(self, x):
        self.out = x @ self.weight
        if self.bias is not None:
            self.out += self.bias
        return self.out
    def parameters(self):
        return [self.weight] + ([self.bias] if self.bias is not None else [])

class BatchNorm1d:
    # ... same as Part 3, will be fixed for 3D inputs in Chapter 19 ...
    pass

class Tanh:
    def __call__(self, x):
        self.out = torch.tanh(x)
        return self.out
    def parameters(self):
        return []

```

These three classes are the toolkit at the start of the lecture. By the end, we will have added three more (`Embedding`, `FlattenConsecutive`, `Sequential`), and the `BatchNorm1d` class will be patched to

handle 3D inputs correctly. That set of six classes is essentially a miniature reimplementation of the relevant pieces of `torch.nn`.

PART II

PyTorch-ifying the code

Building Embedding, Flatten, and Sequential so the whole network is one Sequential(...) call, plus a bug from forgetting eval mode.

4. Fixing the loss plot - the view-and-mean trick

Chapter overview

Before tackling the architectural work, Karpathy spends a few minutes fixing the loss curve plot, which has been ugly since Part 2. With a batch size of 32, single-step losses bounce around by 0.3 or more from one minibatch to the next, producing a noisy curve where it is hard to see what training is actually doing. The fix is one line of PyTorch: reshape the loss list and take the mean along the rows.

Cell - The noisy loss plot (before)

```
plt.plot(lossi) # this looks awful
```

`lossi` is a Python list with 200,000 floats, one for each training step. Plotting it directly produces a near-solid band of noise with a slight downward trend, and a small visible drop at step 150,000 where the learning rate decays. The signal is there but completely buried in noise.

The view-and-mean trick

PyTorch's tensor reshaping gives us a clean way to smooth this. Convert the list to a tensor, reshape it into (200, 1000), and take the mean along the second dimension. The result is a smoothed curve with 200 points, where each point is the average of 1,000 consecutive training losses.

Cell - The smoothed loss plot (after)

```
plt.plot(torch.tensor(lossi).view(-1, 1000).mean(1))
```

- `torch.tensor(lossi)` — convert the Python list to a 1D tensor of shape (200000,).
- `.view(-1, 1000)` — reshape to (200, 1000). The -1 tells PyTorch to figure out the first dimension automatically (200000 / 1000 = 200).
- `.mean(1)` — average along dimension 1 (the second axis, the 1000 consecutive losses), producing a 1D tensor of 200 values.

The resulting plot shows a clear downward trend with the learning-rate decay event clearly visible as a sudden drop. Karpathy emphasizes that this is the version we will use going forward, and that the noisy version is “daggers in his eyes.”

Why this works

Single-step losses on a small batch are very noisy estimates of the true loss, because the 32 examples in any given minibatch can happen to be easy (low loss) or hard (high loss) purely by chance. Averaging across 1,000 consecutive steps effectively averages across 32,000 examples, which is a much more stable estimate. The smoothed curve shows the underlying trend without the per-batch noise. This is also why training-loss curves in papers are almost always smoothed in some way.

5. The Embedding module

Chapter overview

Throughout Parts 2 and 3, the embedding table `C` has been a free-floating tensor, manipulated directly with `C[Xb]`. Karpathy now wraps this into a proper module class that mirrors PyTorch's `nn.Embedding`. The change is mechanical but it cleans up the model construction significantly.

Cell - The Embedding module

```
class Embedding:
    def __init__(self, num_embeddings, embedding_dim):
        self.weight = torch.randn((num_embeddings, embedding_dim))

    def __call__(self, IX):
        self.out = self.weight[IX]
        return self.out

    def parameters(self):
        return [self.weight]
```

The class has the same three-method API as our other modules: a constructor that creates the parameters, a `__call__` method that does the forward computation, and a `parameters` method that returns the trainable tensors. The constructor takes `num_embeddings` (the vocabulary size) and `embedding_dim` (the dimensionality of each character vector), matching PyTorch's signature exactly.

Why this is worth the cosmetic change

After this module exists, the line `emb = C[Xb]` becomes `emb = embedding_layer(Xb)`, which is consistent with how every other layer is called. The embedding now slots into our `Sequential` container (Chapter 7) alongside the other layers, with no special case needed in the forward pass. Conceptually, this also matches the way every modern framework treats embedding tables: as layers, not as naked tensors to be manipulated with indexing.

6. The Flatten module (initial naive version)

Chapter overview

Right after the embedding lookup, the Part 3 code reshapes the result with `emb.view(emb.shape[0], -1)` to flatten each example's three embeddings into a single 30-dimensional vector. Karpathy now wraps this into a `Flatten` module. In this chapter we build the simple version that mirrors PyTorch's `nn.Flatten`. Chapter 14 will generalize it into `FlattenConsecutive`, which is what enables the hierarchical architecture.

Cell - The naive Flatten module (matches PyTorch's `nn.Flatten`)

```
class Flatten:
    def __call__(self, x):
        self.out = x.view(x.shape[0], -1)
        return self.out

    def parameters(self):
        return []
```

The forward pass collapses every dimension except the first (the batch dimension) into a single dimension. For an input of shape (32, 3, 10) representing 32 examples each with 3 characters of 10-dimensional embeddings, the output is (32, 30) — each example becomes a flat 30-dimensional vector ready to feed into a Linear layer. No parameters, no state, just a reshape.

Why `.view()` is cheap

`.view()` never copies tensor data. It just creates a new tensor object that interprets the same underlying memory with different shape metadata. The operation is $O(1)$ regardless of tensor size. This is what makes Flatten essentially free, and it is also what makes the FlattenConsecutive generalization in Chapter 14 efficient even for very large tensors.

7. The Sequential container

Chapter overview

With Embedding and Flatten as proper modules, the entire forward pass of the network is now just “apply this layer, then this layer, then this layer.” That pattern is what `torch.nn.Sequential` exists for. Karpathy implements his own version, and it is one of the smallest pieces of code in the lecture — under ten lines — but it has an outsized effect on how clean the rest of the model definition looks.

Cell - The Sequential container

```
class Sequential:
    def __init__(self, layers):
        self.layers = layers

    def __call__(self, x):
        for layer in self.layers:
            x = layer(x)
        self.out = x
        return self.out

    def parameters(self):
        return [p for layer in self.layers for p in layer.parameters()]
```

- `__init__` — stores the list of layers as a Python list attribute.
- `__call__` — runs the layers in order, threading the output of each into the input of the next, and stores the final result as `self.out`.
- `parameters` — a nested list comprehension that flattens all the child layers' parameter lists into a single list. This is what gets passed to the optimizer.

How the model definition shrinks

Before Sequential, every forward pass had to be written out explicitly:

```
# Before:
emb = C[Xb]
x = emb.view(emb.shape[0], -1)
```



```

for layer in layers:
    x = layer(x)
logits = x
loss = F.cross_entropy(logits, Yb)

```

After Sequential, it collapses to two lines:

```

# After:
logits = model(Xb)
loss = F.cross_entropy(logits, Yb)

```

The Sequential container now contains the embedding layer, the flatten layer, and all the Linear/BatchNorm/Tanh layers. The whole forward pass is one function call. This matches exactly how a model is defined and called in real PyTorch code.

8. The model.layers refactor

Chapter overview

With Sequential in place, the rest of the training loop and the evaluation code need to be updated to access layers through `model.layers` instead of a free-floating `layers` list. This is mechanical but worth showing because it sets up a subtle bug that we hit in the next chapter.

Cell - The new model construction

```

torch.manual_seed(42)

n_embd = 10
n_hidden = 200

model = Sequential([
    Embedding(vocab_size, n_embd),
    Flatten(),
    Linear(n_embd * block_size, n_hidden, bias=False),
    BatchNorm1d(n_hidden),
    Tanh(),
    Linear(n_hidden, vocab_size),
])

with torch.no_grad():
    model.layers[-1].weight *= 0.1  # last layer less confident

parameters = model.parameters()
for p in parameters:
    p.requires_grad = True

```

The whole network is now a single `Sequential(...)` call. The output-layer dampening from Part 3 (multiplying by 0.1 to fix the confidently-wrong initialization) is now done by reaching into `model.layers[-1]`. The parameter list is collected by calling `model.parameters()`, and the rest of the training loop is unchanged.

Cell - The training loop, simplified

```

max_steps = 200000

```

```

batch_size = 32
lossi = []

for i in range(max_steps):
    ix = torch.randint(0, Xtr.shape[0], (batch_size,))
    Xb, Yb = Xtr[ix], Ytr[ix]

    logits = model(Xb)
    loss = F.cross_entropy(logits, Yb)

    for p in parameters:
        p.grad = None
    loss.backward()

    lr = 0.1 if i < 150000 else 0.01
    for p in parameters:
        p.data += -lr * p.grad

    if i % 10000 == 0:
        print(f'{i:7d}/{max_steps:7d}: {loss.item():.4f}')
    lossi.append(loss.log10().item())

```

Everything that was custom forward-pass code has collapsed into the single line `logits = model(Xb)`. The rest is the same training loop we have had since Part 2: random minibatch, forward, backward, update.

9. A bug from forgetting eval mode

Chapter overview

While refactoring, Karpathy hits a subtle bug that is worth understanding: the BatchNorm layer produces nonsense when given a single example because the variance of a single number is undefined. The bug surfaces when sampling from the model, where each step forwards a batch of size one through the network. This chapter explains the failure, the diagnosis, and the fix.

What goes wrong

During training, BatchNorm computes the mean and variance of its input across the batch dimension. With a batch of 32 examples, this works fine. During sampling, we forward one context at a time — a batch of size one. The mean of a single number is just that number, and the variance of a single number is undefined (you cannot estimate spread from one observation; the denominator $n - 1$ is zero). PyTorch returns NaN, which then propagates through every downstream computation. The sampled output becomes garbage.

Karpathy diagnosing live

"I fixed the issue and we now get the result that we expect, which is gibberish because the model is not trained. The problem was that when I fixed this cell to be `model.layers` instead of just `layers`, I did not actually run the cell, and so our neural net was in training mode. Then we are passing in an input which is a batch of just a single example, and so the variance of a single number is not a number."

The fix

Before sampling, set every layer's training flag to `False`. This tells BatchNorm to use its running statistics (accumulated during training) instead of trying to compute fresh statistics from a single example. The same toggle is needed before evaluating loss on the dev or test split, since those evaluations also happen with the model in inference mode.

Cell - Switching to eval mode

```
# put layers into eval mode (needed for batchnorm especially)
for layer in model.layers:
    layer.training = False
```

The general lesson

Anything stateful in a deep learning framework is a bug waiting to happen. BatchNorm's train/eval flag is the most common offender, but dropout has the same issue, as do certain kinds of attention layers. The convention to wrap all of this in `model.eval()` and `model.train()` is exactly so you do not have to remember which layers care. In our miniature framework we are doing the toggle manually because we have not implemented those container methods yet.

PART III

Scaling context and the matmul insight

Block size from 3 to 8, why a single hidden layer cannot make good use of a long context, and the matrix-multiplication fact that makes the hierarchical fix possible.

10. Increasing block size from 3 to 8

Chapter overview

Before changing the architecture, we change one hyperparameter: the context length. Three characters is a tight window — the model trying to predict the next letter of 'amelie' only ever gets to see 'eli', not the full name. Eight characters gives the network a much richer view. Before we even touch the architecture, just increasing the block size produces a meaningful improvement.

The change is one line in `build_dataset`: `block_size = 8`. Everything else — the data preprocessing, the model definition, the training loop — carries over unchanged. The first linear layer is now `Linear(n_embd * 8, n_hidden)` instead of `Linear(n_embd * 3, n_hidden)`, so the parameter count grows from about 12K to about 22K.

11. The baseline result of lazily scaling context

Chapter overview

Karpathy trains the new 8-character model with the existing flat architecture and records the result. This is the “just scale the context” baseline that any architectural improvement has to beat. The result is meaningful: 0.08 of loss reduction from a single hyperparameter change.

The result

Configuration	Params	Train loss	Val loss
Part 3 baseline (3-char context, 200 hidden)	~12K	2.058	2.105
Context 3 -> 8 (still flat)	~22K	1.918	2.027

The validation loss drops from 2.105 to 2.027 just from giving the network more context. The training loss drops more aggressively (from 2.058 to 1.918), which suggests the larger model has more capacity to memorize the training data — but we are not overfitting catastrophically because the val loss also improved meaningfully. The samples produced are noticeably more name-like.

What this tells us

“Just scaling up the context length gives us quite a bit of an improvement. So we could of course spend a lot of time here tuning things and making it even bigger and scaling up the network further. But let's continue and let's implement the hierarchical scheme and treat this as just a rough baseline performance — there's a lot of optimization left on the table.”

12. Why crushing information too fast is suboptimal

Chapter overview

With the baseline in hand, we return to the conceptual problem from Chapter 1. The flat 8-character model takes the (32, 8, 10) embedding tensor, flattens it to (32, 80), and immediately multiplies by an 80×200 weight matrix. Every hidden neuron now depends on every character of the context in one shot. This chapter explains why that is wasteful even though it works.

The squashing problem, made concrete

Consider what we are asking the first hidden layer to do. It sees a flat vector of 80 numbers containing the embeddings of eight characters concatenated end-to-end. From those 80 numbers, it has to learn:

- Which characters are next to each other (positions 0-9 are character 1, positions 10-19 are character 2, etc).
- What bigrams those neighboring characters form (whether 'th' or 'qu' or 'xz' is locally meaningful).
- Whether those bigrams combine into longer patterns ('the', 'eri', 'ann').
- How all of that interacts to determine the next character.

All four levels of structure have to be learned in a single matrix multiplication, with no architectural prior to guide the network. The hidden representations after that first layer are an unconstrained 200-dimensional summary of everything, and there is no guarantee that the information is organized in any particular way. Subsequent layers cannot recover structure that the first layer chose to discard.

The compression metaphor

Think of the first hidden layer as a compression bottleneck. The network has to encode everything it knows about the eight-character context into 200 numbers, and it has to do this in one step. A hierarchical architecture, by contrast, compresses gradually: first into four bigram representations of dimension 128 (so 512 numbers total), then into two four-gram representations (256 numbers), then into one eight-gram representation (128 numbers). At each stage the network only has to make a small compression decision, and it can preserve structure that a single-step compression would have to throw away.

Why the hierarchical model promises something better

The hierarchical architecture builds in the prior that neighboring characters are more related to each other than to distant characters. The first layer only fuses pairs of *adjacent* embeddings, not all eight at once. The bigram representations it produces are interpretable in a way that the flat hidden state is not. Subsequent layers similarly fuse pairs of adjacent bigrams into four-grams, and so on. The network is not forced to learn the “adjacency matters” prior from scratch — the architecture provides it for free.

13. The matmul broadcasting insight

Chapter overview

Before we can build the hierarchical model, we need a small but powerful fact about how matrix multiplication interacts with extra dimensions in PyTorch. This chapter establishes the fact, which Karpathy explicitly calls “surprising” even though it is documented behaviour. The insight is what makes the FlattenConsecutive design possible.

The setup

In our existing flat architecture, the input to the first Linear layer has shape (32, 80) and the weight matrix has shape (80, 200). The output is (32, 200). The matmul is two-dimensional. Standard.

Here is the surprising fact: PyTorch's matmul operator actually accepts more dimensions on the input. If the input has shape (32, 4, 80) instead of (32, 80), and the weight is still (80, 200), the matmul produces a (32, 4, 200) output. The matmul only operates on the last dimension; all the dimensions before it are treated as batch.

Cell - Demonstrating last-dim-only matmul

```
# Standard 2D matmul:
x = torch.randn(32, 80)
W = torch.randn(80, 200)
(x @ W).shape    # torch.Size([32, 200])

# Same weight, 3D input - matmul still works:
x = torch.randn(32, 4, 80)
(x @ W).shape    # torch.Size([32, 4, 200])

# Same weight, 4D input - still works:
x = torch.randn(32, 4, 5, 80)
(x @ W).shape    # torch.Size([32, 4, 5, 200])
```

The rule, stated precisely

When you compute `x @ W`, the matrix multiplication is performed only on the last two dimensions of `x` with `W`. Every dimension of `x` before the last two is treated as a batch dimension and is preserved unchanged in the output. The same Linear layer can therefore be applied to data with any number of leading batch dimensions, as long as the last dimension matches the layer's `fan_in`.

Why this matters for the hierarchical model

This is the key fact that makes the WaveNet-like architecture cheap to build. We can reshape the 8-character embedding tensor from (32, 8, 10) into (32, 4, 20) — four pairs of consecutive embeddings, each pair flattened into a 20-dimensional vector — and then pass the result through a single Linear(20, `n_hidden`) layer. The Linear layer operates *independently* on each of the four pair positions, in parallel, with shared weights. All four bigram fusions happen in one matrix multiplication, which is exactly what we want.

The shared-weights interpretation

Notice that the same weight matrix is used to fuse all four pairs. This is weight sharing, exactly like in a convolution. The network learns one bigram-fusing function and applies it at every position in the context. This is also what makes the hierarchical model parameter-efficient: it has one weight matrix per layer instead of one per position.

14. The FlattenConsecutive generalization

Chapter overview

Armed with the matmul insight, we can now build the module that makes the hierarchical architecture work. `FlattenConsecutive(n)` takes a tensor of shape (B, T, C) and packs every n consecutive positions along T into the last dimension, producing a tensor of shape $(B, T/n, C*n)$. This is the reshape that aligns adjacent embeddings into pairs (or any other grouping) so that a Linear layer can fuse them with the matmul broadcasting from Chapter 13.

Cell - The FlattenConsecutive module

```
class FlattenConsecutive:
    def __init__(self, n):
        self.n = n

    def __call__(self, x):
        B, T, C = x.shape
        x = x.view(B, T // self.n, C * self.n)
        if x.shape[1] == 1:
            x = x.squeeze(1)
        self.out = x
        return self.out

    def parameters(self):
        return []
```

Reading the forward pass

- $B, T, C = x.shape$ — unpack into batch, time/positions, channels. For our 8-character context with 10-dimensional embeddings, this is $(32, 8, 10)$.
- $x.view(B, T // n, C * n)$ — reshape. With $n = 2$, this is $(32, 8/2, 10*2) = (32, 4, 20)$. Four pairs, each represented by a flat 20-dimensional concatenation of two embeddings.
- `if x.shape[1] == 1: x = x.squeeze(1)` — the spurious-dimension handler. Once we have fused everything into a single position, the middle dimension becomes 1; we remove it so the result can feed into a 2D Linear layer.

Why `.view()` works for this reshape

`.view()` reinterprets the underlying memory layout. PyTorch tensors are stored row-major, so the (8, 10) part of the (32, 8, 10) tensor is laid out as 80 consecutive floats per example: positions 0-9 are character 1's embedding, positions 10-19 are character 2's, and so on. Viewing as (4, 20) groups positions 0-19 as the first 20-dim vector (characters 1 and 2 concatenated), 20-39 as the second (characters 3 and 4), and so on. The grouping by pairs happens automatically because of how the memory is laid out. No explicit concatenation needed.

Karpathy verifies this in the lecture by also writing the explicit version — selecting even-indexed positions, odd-indexed positions, and concatenating them — and confirms that `.view()` produces exactly the same tensor. The view-based form is much cleaner and entirely free, since no data is copied.

Why this is named `FlattenConsecutive` and not `Flatten`

PyTorch already has a `torch.nn.Flatten` class with different semantics: it flattens everything into one dimension or operates on a specified range of dimensions. Karpathy explicitly renames our class to `FlattenConsecutive` so it does not conflict with PyTorch's API. The name is also descriptive: we are flattening *n consecutive* positions into the last dimension, not flattening everything.

Why the squeeze is there

After three `FlattenConsecutive(2)` stages applied to an 8-position context, the shapes go (B, 8, C) -> (B, 4, 2C) -> (B, 2, 4C) -> (B, 1, 8C). The final stage produces a tensor with a middle dimension of 1, which is technically correct but is now equivalent to a 2D (B, 8C) tensor. The squeeze removes that spurious dimension so the final Linear layer sees a familiar 2D input. Without it, the final Linear would still work (because of the matmul broadcasting from Chapter 13), but the output shape would be (B, 1, vocab_size), and downstream operations like cross-entropy would need an extra reshape.

PART IV

The hierarchical architecture and the BatchNorm bug

Stacking three FlattenConsecutive stages, discovering that BatchNorm was silently broken for 3D inputs, and finally crossing the 2.0 mark.

15. The squeeze trick for the final flatten

Chapter overview

Chapter 14 introduced the squeeze logic inside FlattenConsecutive. This chapter zooms in on exactly why it is needed and what would happen without it, because the squeeze is the smallest piece of the module but it is the most surprising one. If you skipped over it the first time, this chapter is the closer look.

What the shapes look like across the stack

For an 8-character context with 10-dimensional embeddings, the tensor shapes through three FlattenConsecutive(2) stages are:

Stage	Before FlattenConsecutive(2)	After FlattenConsecutive(2)
Stage 1 (chars -> bigrams)	(B, 8, 10)	(B, 4, 20)
Stage 2 (bigrams -> four-grams)	(B, 4, hidden)	(B, 2, 2*hidden)
Stage 3 (four-grams -> eight-gram)	(B, 2, hidden)	(B, 1, 2*hidden) -> (B, 2*hidden)

Stages 1 and 2 produce sensible three-dimensional outputs. Stage 3 is different: after fusing the last pair, there is only one position left in the middle dimension. The squeeze removes that singleton dimension so the result is a clean 2D (B, 2*hidden) tensor, ready to feed the final Linear classifier.

Without the squeeze

If we forgot the squeeze, the final layer would produce a tensor of shape (B, 1, vocab_size). Technically still correct, but every downstream use (cross-entropy, softmax, argmax) would need an explicit reshape or would broadcast in unexpected ways. The squeeze is a small price to pay to keep the rest of the code clean. Karpathy makes this decision explicit: be very explicit about what dimensions are being removed, so that errors surface immediately if the input shape is not what you expect.

16. Stacking three FlattenConsecutive + Linear + BatchNorm + Tanh

Chapter overview

With FlattenConsecutive in hand, we can finally write the hierarchical model. The architecture is three identical stages followed by an output projection. Each stage takes the current representation, fuses pairs

of adjacent positions, projects them into a new hidden space, normalizes the activations, and applies a tanh non-linearity. This chapter presents the model and walks through what each stage is doing semantically.

Cell - The hierarchical model

```
model = Sequential([
    Embedding(vocab_size, n_embd),
    FlattenConsecutive(2), Linear(n_embd * 2, n_hidden, bias=False), BatchNorm1d(n_hidden), Tanh(),
    FlattenConsecutive(2), Linear(n_hidden * 2, n_hidden, bias=False), BatchNorm1d(n_hidden), Tanh(),
    FlattenConsecutive(2), Linear(n_hidden * 2, n_hidden, bias=False), BatchNorm1d(n_hidden), Tanh(),
    Linear(n_hidden, vocab_size),
])
```

Stage by stage

- **Embedding(vocab_size, n_embd)** — convert the 8 integer character indices into 8 dense vectors. Input (B, 8), output (B, 8, n_embd).
- **Stage 1: FlattenConsecutive(2) -> Linear -> BN -> Tanh** — fuse pairs of adjacent character embeddings into bigram representations. The Linear layer's input width is $n_embd * 2$ because each pair contributes two embeddings. Output (B, 4, n_hidden).
- **Stage 2: same pattern** — fuse pairs of adjacent bigrams into four-gram representations. The Linear layer's input width is $n_hidden * 2$ because each pair contributes two bigram representations. Output (B, 2, n_hidden).
- **Stage 3: same pattern** — fuse the two four-grams into a single eight-gram representation. After the FlattenConsecutive(2), the middle dimension is 1; the squeeze removes it. The Linear and BN now operate on 2D input. Output (B, n_hidden).
- **Linear(n_hidden, vocab_size)** — project the final eight-gram representation into 27 logits. The standard classification head.

The tree structure, finally visible

Look at the model definition. Three identical stages, each fusing pairs. The shape of the computation is a binary tree: 8 characters -> 4 bigrams -> 2 four-grams -> 1 eight-gram. Every output position at any level sees a power-of-two-sized window of input characters. This is the WaveNet topology, expressed in our minimal layer library.

17. Inspecting shapes through the network

Chapter overview

When you change architectures, the first thing to do is forward a single batch through and print every layer's output shape. This is the basic sanity check that nothing is silently broken. Karpathy walks through this exercise in the lecture, and it is worth doing slowly because the shapes are the load-bearing detail of the whole design.

Cell - Forward a batch and inspect every layer's shape

```
# Forward a small batch and print each layer's output shape
ix = torch.randint(0, Xtr.shape[0], (4,))
Xb, Yb = Xtr[ix], Ytr[ix]
logits = model(Xb)

for layer in model.layers:
    print(layer.__class__.__name__, ': ', tuple(layer.out.shape))
```

What you should see

For an input batch of 4 examples with block_size 8 and embedding dim 10, the printed shapes should be:

Layer	Output shape
Embedding	(4, 8, 10)
FlattenConsecutive(2)	(4, 4, 20)
Linear	(4, 4, 128)
BatchNorm1d	(4, 4, 128)
Tanh	(4, 4, 128)
FlattenConsecutive(2)	(4, 2, 256)
Linear	(4, 2, 128)
BatchNorm1d	(4, 2, 128)
Tanh	(4, 2, 128)
FlattenConsecutive(2)	(4, 256) # squeezed
Linear	(4, 128)
BatchNorm1d	(4, 128)
Tanh	(4, 128)
Linear	(4, 27)

Each row is one layer in the Sequential. The pattern is clear: after each stage of Flatten/Linear/BN/Tanh, the time dimension halves and the channel dimension stays at n_hidden. The third FlattenConsecutive(2) triggers the squeeze because the time dimension would otherwise become 1. The final Linear projects from n_hidden to vocab_size = 27.

Why inspect every shape

Multi-dimensional networks are riddled with shape bugs. A misaligned dimension can produce an output that is technically valid (and may even train to a loss that looks plausible) but is doing the wrong computation. The only defense is to print shapes after every layer the first time you build a new architecture. If a shape is unexpected — like a two-dimensional tensor where you expected three-dimensional — track it down before continuing. This is exactly how Karpathy will catch the BatchNorm bug in Chapter 19.

18. Hierarchical with same parameter count: no improvement yet

Chapter overview

Karpathy is careful to control for parameter count when comparing architectures. He sets `n_hidden = 68` in the hierarchical model so that the total parameter count comes out to about 22K, matching the flat 8-character baseline from Chapter 11. This is the right comparison: same capacity, different structure, which model uses it better?

The result, before bug fix

Architecture (22K params)	Train loss	Val loss
Flat 8-char context, 1 hidden layer	1.918	2.027
Hierarchical 3-stage (<code>n_hidden=68</code>)	1.941	2.029

Essentially identical performance. The hierarchical model is not (yet) better than the flat one at the same parameter count. This is the moment in the lecture where Karpathy could have concluded “hierarchical structure doesn't help” and stopped. He does not. Instead he raises the question: is the BatchNorm doing what we think it is doing in this new 3D-input regime? The next chapter shows that the answer is no.

Karpathy's diagnosis impulse

“Controlling for the number of parameters, changing from the flat to hierarchical is not giving us anything yet. That said, we didn't really torture the architecture here very much — this is just my first guess. And number two: we still may have a bug inside the BatchNorm1d layer.” This is the right instinct. If a structural change that should help is not helping, suspect a bug before you suspect the theory.

19. The BatchNorm 3D bug: reducing over the wrong dimension

Chapter overview

Our BatchNorm1d from Part 3 was written under the assumption that the input is 2D — shape (batch, channels). In the hierarchical model the BatchNorm layers in stages 1 and 2 receive 3D inputs of shape (batch, time, channels). The layer does not error out because everything broadcasts correctly, but the statistics it computes are wrong in a subtle way. This chapter walks through the bug, the diagnosis (which uses the same shape-inspection discipline from Chapter 17), and the fix.

How to spot the bug

The diagnostic is to print the shape of `running_mean` on one of the BatchNorm layers after a forward pass. It should be a 1D tensor of length `n_hidden` — one running mean per channel, as the layer is supposed to maintain. Instead, with our buggy implementation, the shape is (1, 4, `n_hidden`) in stage 1 or (1, 2, `n_hidden`) in stage 2. We are maintaining a separate set of statistics for each time position rather than sharing across positions.

Cell - Diagnosing the BatchNorm bug

```
# After one forward pass, check the running_mean shape:
print(model.layers[3].running_mean.shape)
# Buggy: torch.Size([1, 4, 68])      -- separate stats per position!
# Fixed:  torch.Size([1, 1, 68])      -- shared stats across positions
```

Why the bug is silent

The forward pass with the buggy BatchNorm produces tensors of the right shape because the mean/std subtraction broadcasts correctly with the (1, T, C) running statistics. The network trains. The loss looks plausible. The bug only shows up if you specifically look at the shape of the running statistics and realize that the layer is treating each time position as an independent channel rather than batching them together.

The root cause

Our BatchNorm1d takes the mean over dim 0 only, with `x.mean(0, keepdim=True)`. For a 2D input of shape (batch, channels), this correctly produces (1, channels), reducing across the batch. For a 3D input of shape (batch, time, channels), this only reduces across the batch dimension, leaving the time dimension intact and producing (1, time, channels). The result is that each time position gets its own mean and variance, computed from only the `batch_size` examples at that position. We want to also reduce across time, computing one mean and one variance per channel from all `batch * time` samples.

The fix

The fix is to detect whether the input is 2D or 3D and reduce over the appropriate dimensions. For 2D input, reduce over dim 0. For 3D input, reduce over dims (0, 1) — both the batch and the time dimensions

simultaneously.

Cell - The corrected BatchNorm1d

```
class BatchNorm1d:
    def __init__(self, dim, eps=1e-5, momentum=0.1):
        # ... same as before ...
        pass

    def __call__(self, x):
        if self.training:
            if x.ndim == 2:
                dim = 0
            elif x.ndim == 3:
                dim = (0, 1)
            xmean = x.mean(dim, keepdim=True)
            xvar = x.var(dim, keepdim=True)
        else:
            xmean = self.running_mean
            xvar = self.running_var
        xhat = (x - xmean) / torch.sqrt(xvar + self.eps)
        self.out = self.gamma * xhat + self.beta
        if self.training:
            with torch.no_grad():
                self.running_mean = (1 - self.momentum) * self.running_mean + self.momentum * xmean
                self.running_var = (1 - self.momentum) * self.running_var + self.momentum * xvar
        return self.out

    def parameters(self):
        return [self.gamma, self.beta]
```

The key lines are the conditional on `x.ndim`. PyTorch's `tensor.mean(dim, keepdim=True)` accepts either an integer or a tuple of integers as the `dim` argument. Passing a tuple `(0, 1)` reduces over both dimensions simultaneously. The result is a tensor with two singleton leading dimensions, exactly what we want for broadcasting in the normalize step.

The improvement

Configuration	Train loss	Val loss
Hierarchical, buggy BN	1.941	2.029
Hierarchical, fixed BN	1.912	2.022

A small but real improvement (about 0.007 on val), and we now have many more samples contributing to each statistic estimate. With `batch_size = 32` and `time = 4`, each running mean is now estimated from 128 samples per step instead of 32. The estimates are more stable, and the network trains slightly better as a result. Karpathy notes that this is not necessarily statistically significant from a single run, but the change is principled.

20. PyTorch's BatchNorm1d convention (NxCxL vs N×L×C)

Chapter overview

Now that our BatchNorm handles 3D inputs, it is worth pointing out that we are doing it differently from PyTorch's stock `nn.BatchNorm1d`. PyTorch expects the channel dimension in the middle: (N, C, L) where N is batch, C is channels, L is length. Our convention is (N, L, C) — channels last. This chapter explains the difference and why our convention is the one Karpathy prefers.

PyTorch's convention

PyTorch's `nn.BatchNorm1d` accepts either (N, C) for 2D inputs or (N, C, L) for 3D inputs. In the 3D case, the channel dimension is always in position 1 and the time dimension is in position 2. This convention is inherited from computer vision, where image tensors are (N, C, H, W) — channels first — and the convolutional stack runs end-to-end with that layout.

Our convention

We use (N, L, C) instead. The channel dimension is always last. The leading dimensions are all batch-like and are reduced over when computing statistics. The advantage is uniformity: the `matmul` broadcasting from Chapter 13 works on the last dimension, BatchNorm normalizes along the last dimension, every layer treats “channels” the same way. The downside is that we are not API-compatible with PyTorch's `BatchNorm1d` if someone tries to drop our model into a larger PyTorch system.

Why this matters in practice

“I think it's okay — I prefer it this way honestly. So this is the way that we will keep it for our purposes.” The Transformer literature also uses channels-last (N, L, C), so our convention is more in line with what modern NLP code looks like. PyTorch's `BatchNorm1d` is somewhat out of step with the rest of the framework's recent conventions. The API mismatch is a real annoyance if you build a model in our style and then want to use `torch.nn.BatchNorm1d` as a drop-in replacement — you have to transpose the input.

21. Scaling up: n_embd 24, n_hidden 128, crossing 2.0

Chapter overview

With the architecture in place and the BatchNorm bug fixed, Karpathy makes the final scaling decision: bump the embedding dimension from 10 to 24 and the hidden width from 68 to 128. Same three-stage architecture, just bigger. This is what finally crosses the 2.0 validation loss mark that has stood since Part 2.

Cell - The scaled-up model

```
n_embd = 24    # was 10
n_hidden = 128 # was 68
```

```
model = Sequential([
    Embedding(vocab_size, n_embd),
    FlattenConsecutive(2), Linear(n_embd * 2, n_hidden, bias=False), BatchNorm1d(n_hidden), Tanh(),
    FlattenConsecutive(2), Linear(n_hidden * 2, n_hidden, bias=False), BatchNorm1d(n_hidden), Tanh(),
    FlattenConsecutive(2), Linear(n_hidden * 2, n_hidden, bias=False), BatchNorm1d(n_hidden), Tanh(),
    Linear(n_hidden, vocab_size),
])
```

```
# Total: about 76K parameters
```

The result

Configuration	Params	Train loss	Val loss
Hierarchical fixed BN (n_hidden=68)	~22K	1.912	2.022
Scaled up (n_embd=24, n_hidden=128)	~76K	1.769	1.993

1.993 validation loss. First time the series has crossed below 2.0. The improvement from 2.022 to 1.993 is real and reproducible. The training loss also drops substantially, from 1.912 to 1.769, suggesting we have headroom for further capacity scaling without overfitting becoming a problem.

Honest framing from Karpathy

“We’ve crossed over the 2.0 sort of territory and right about 1.99, but we are starting to have to wait quite a bit longer, and we’re a little bit in the dark with respect to the correct setting of the hyperparameters here and the learning rates and so on, because the experiments are starting to take longer to train. We are missing sort of like an experimental harness on which we could run a number of experiments and really tune this architecture very well.” The 1.993 is real, but not optimized; there is likely more to be found here by anyone willing to do a proper hyperparameter sweep.

PART V

Convolutions, takeaways, and reference

The convolution preview that points to the next lecture, the full performance log across the series, and the glossary.

22. The convolution-is-a-for-loop insight

Chapter overview

We have built the WaveNet topology, but we are not using convolutions to implement it. Karpathy closes the lecture by previewing how this hierarchical model relates to the convolutional networks the original WaveNet paper actually uses. The key insight is that convolution is “a for loop” that slides our existing model across the input sequence. The architecture does not change — convolutions just make it faster.

What our model does for a single output position

Take the name 'DeAndre' from the training set. It is seven letters, which produces eight training examples (each context-target pair, plus the final stop token). For each of those eight examples, we currently call our model once on a 1×8 input and get one logits vector back. Eight independent forward passes through the same model.

Cell - Forwarding each example independently

```
# Eight separate forwards, one per training example for 'DeAndre':
logits = torch.zeros(8, 27)
for i in range(8):
    logits[i] = model(Xtr[[7 + i]])
```

What convolutions do instead

A convolution takes our model (which produces one output from one context) and slides it across the input sequence, producing all output positions in parallel. The for loop above moves inside the convolution operator and runs inside optimized CUDA kernels instead of Python. The model itself is unchanged.

Convolutions as Karpathy framed it

“What convolutions allow you to do is to take this black structure and kind of like slide it over the input sequence here and calculate all of these orange outputs at the same time. The convolution basically — you can think of it — is a for loop applying a little linear filter over space of some input sequence.” The hierarchical model we built is exactly such a “little linear filter” (well, three of them stacked). Convolutions are the machinery for applying it efficiently across many positions.

23. Variable reuse and receptive field overlap

Chapter overview

Beyond just being faster, convolutions exploit a property our naive implementation cannot: when overlapping windows share input positions, the intermediate computations on those shared positions can be reused across windows. This chapter walks through the savings concretely.

Where the overlap comes from

Look at two consecutive output positions in our model. To predict the character at position 8, the model uses characters at positions 0-7. To predict the character at position 9, it uses characters at positions 1-8. The two windows share seven of their eight characters. The first FlattenConsecutive(2) stage's outputs for positions (1, 2), (3, 4), (5, 6) are identical between the two windows. Computing them twice is wasted work.

The savings, made concrete

“If we look at this node here — this node here is the right child of this node but is also the left child of the node here. And so basically this node and its value is used twice, and so right now in this naive way we would have to recalculate it. But here we are allowed to reuse it.” A convolutional implementation only computes each intermediate value once and reuses it across all output positions that need it. For long sequences this is a big win.

Our naive implementation forwards each of the eight examples independently and never reuses anything. A convolutional implementation would forward the entire name 'DeAndre' as a single 1×8 input and produce eight output logits in one pass, sharing the intermediate computations across positions. Same model, same final outputs, much less compute.

24. What WaveNet's full architecture adds

Chapter overview

We borrowed only the tree-like topology from WaveNet. The full paper layers several other ideas on top, all designed to make very deep stacks train well. This chapter briefly lists what the full WaveNet uses and what each ingredient is for. None of it is implemented in our lecture, but knowing what is out there helps you read the paper if you want to.

- **Dilated causal convolutions.** The tree-like structure expressed using convolutions with progressively larger gaps between input positions. Each layer doubles the gap, achieving exponential receptive field growth using only fixed-size filters. The “causal” part ensures that the prediction at position t only depends on positions 0 through $t-1$, never on future positions.
- **Gated activation units.** Instead of a simple $\tanh(x)$, WaveNet uses $\tanh(x_a) * \text{sigmoid}(x_b)$, where x_a and x_b are two separate linear projections. The sigmoid acts as a learned gate that decides how much of the tanh output to let through. Borrowed from the LSTM family.
- **Residual connections.** Each layer's output is the layer's transformation plus the layer's input, instead of just the transformation. Lets gradients flow more directly to early layers and stabilizes training of very deep stacks. Without residual connections, networks beyond about 20 layers were practically untrainable before 2015.
- **Skip connections.** Outputs from intermediate layers are added together and contribute directly to the final prediction. Lets the final logits draw on features from many depths, not just the deepest one.

- **Conditioning.** WaveNet for audio is conditioned on global context (speaker identity, language) and local context (linguistic features at each timestep). These conditioning signals are mixed into every layer.

Why we skipped all of this

Our network is only three stages deep and we are predicting characters, not audio. We do not need residual connections because gradients can already flow through three layers without trouble. We do not need gated activations because plain tanh works fine at this scale. We do not need conditioning because we are not modelling speakers or languages. The full WaveNet machinery is necessary at the scale and complexity of the original paper, but for our purposes the topology alone is enough to learn the lesson.

25. The development process: docs, shapes, Jupyter prototyping

Chapter overview

Karpathy spends the last few minutes of the lecture reflecting on what the actual development process for deep learning code looks like. He is unusually candid about three pain points: PyTorch documentation is unreliable, shape gymnastics consumes a lot of time, and the Jupyter-to-VS-Code workflow is essentially universal. This chapter captures those reflections because they are some of the most practically useful advice in the entire series.

On PyTorch documentation

Karpathy's verdict: "PyTorch's documentation is not very good. They spend a ton of time on hardcore engineering of all kinds of distributed primitives etc, but as far as I can tell no one is maintaining any documentation. It will lie to you, it will be wrong, it will be incomplete, it will be unclear. So unfortunately it is what it is and you just kind of do your best with what they've given us." The practical implication: do not assume documentation is correct. Read the source code of the layer you are using when in doubt. Run small experiments to verify behaviour before relying on it.

On shape gymnastics

"There's a ton of trying to make the shapes work and there's a lot of gymnastics around these multi-dimensional arrays. Are they two-dimensional, three-dimensional, four-dimensional? What layers take what shapes? Is it NCL or NLC? And you're permuting and viewing and it just can get pretty messy." This is universal experience for everyone who works with deep learning code. The defense Karpathy recommends is to print shapes constantly while you are building a new model — exactly the discipline he demonstrates in Chapter 17.

On the Jupyter-to-VS-Code workflow

"I very often prototype these layers and implementations in Jupyter notebooks and make sure that all the shapes work out and I'm spending a lot of time basically babysitting the shapes and making sure everything is correct. And then once I'm satisfied with the functionality in the Jupyter notebook I will take that code and copy paste it into my repository of actual code that I'm training with." This is the working pattern of most deep learning practitioners: Jupyter for prototyping and shape-checking, a real code repository for training runs. Notebooks are for figuring out what works; scripts are for scaling it up.

Three concrete habits to take away

1. Always print shapes after every layer when building a new architecture. 2. Do not trust PyTorch's documentation completely; verify with small experiments. 3. Develop in Jupyter to iterate on shapes and behaviours, then move polished code into a proper repository for training. These are the habits that separate productive deep learning practitioners from people who flail.

26. The full performance log across the series

Chapter overview

This chapter consolidates the validation loss progression across the entire makemore series so far. The table tells the story of five lectures' worth of work: from the bigram model in Part 1 to the WaveNet-style hierarchical model in Part 5, the validation loss has fallen by about 25%. Each row is one concrete intervention; each column is the lecture in which it appeared.

Model / Intervention	Lecture	Val loss
Bigram model (counting only)	Part 1	2.45
MLP (3-char context, 200 hidden)	Part 2	2.17
+ fix confidently-wrong init	Part 3	2.13
+ fix tanh saturation	Part 3	2.10
+ principled Kaiming init	Part 3	2.11
+ BatchNorm	Part 3	2.10
+ manual backprop (no change in loss)	Part 4	2.12
+ block_size 3 -> 8	Part 5	2.03
+ hierarchical (same params)	Part 5	2.03
+ BatchNorm 3D bug fix	Part 5	2.02
+ scale up (76K params)	Part 5	1.99

Five interventions in this lecture alone contributed to the cumulative improvement, and the two biggest contributors were the block-size increase (about 0.08 of loss) and the scaling-up (about 0.03 of loss). The hierarchical structure on its own, at matched parameter count, did not help — but it set up the conditions for scaling that the flat architecture could not have supported as gracefully.

27. Diagnostic cards: shape and BatchNorm pitfalls

Chapter overview

The bugs that come up when working with multi-dimensional tensors and stateful layers fall into a small number of patterns. This chapter consolidates the ones from this lecture into stand-alone diagnostic cards. Keep these handy — you will hit every one of them at some point in your own work.

Pitfall 1 · BatchNorm reducing over wrong dimension

Symptom	<code>running_mean</code> has an unexpected extra dimension. Hierarchical model with BatchNorm trains but does not improve over the flat baseline at matched parameter count.
Diagnosis	BatchNorm written for 2D inputs is given 3D inputs. The mean and var reduce over dim 0 only, leaving the time dimension intact. Each time position gets its own running statistics instead of sharing across positions.
Action	Detect <code>x.ndim</code> and choose the reduction dimensions accordingly: dim 0 for 2D, (0, 1) for 3D. Verify by checking that <code>running_mean.shape</code> has all-singleton leading dimensions.

Pitfall 2 · BatchNorm in training mode at inference

Symptom	Sampling produces gibberish even after training. Loss values when evaluating on dev/test are inconsistent or wildly off.
Diagnosis	BatchNorm uses batch statistics during training and running statistics at inference. Forgetting to switch off training mode means BatchNorm tries to compute statistics from a single example, producing NaN (since variance of a single number is undefined).
Action	Always set <code>layer.training = False</code> for every layer before sampling or evaluating loss. PyTorch's <code>model.eval()</code> handles this automatically; in our miniature framework we toggle manually.

Pitfall 3 · Confusing $N \times C \times L$ vs $N \times L \times C$

Symptom	Shape error when trying to combine your code with PyTorch's <code>nn.BatchNorm1d</code> or <code>nn.Conv1d</code> . Or worse, no error but the layer normalizes over the wrong dimension.
Diagnosis	PyTorch's stock layers expect channels-first (N, C, L). Our convention is channels-last (N, L, C). Dropping one in for the other without a transpose silently does the wrong thing.
Action	Pick one convention and use it consistently. If you need to use PyTorch's layers with our convention, transpose before and after: <code>x.transpose(1, 2)</code> .

Pitfall 4 · Squeeze missing on final flatten

Symptom	Final layer's output has shape (B, 1, vocab_size) instead of (B, vocab_size). Cross-entropy or downstream code complains about an unexpected dimension.
Diagnosis	<code>FlattenConsecutive(n)</code> called when the time dimension is already n (the last possible step in a hierarchical model) produces a tensor with a singleton middle dimension that confuses downstream layers.
Action	The squeeze inside <code>FlattenConsecutive</code> handles this automatically. If you write a custom flatten and forget the squeeze, the bug will show up as either a shape error in cross-entropy or unexpected broadcasting in the output layer.

Pitfall 5 · Noisy loss curve

Symptom	Loss plot looks like a solid band of noise with only a slight downward trend visible. Cannot tell whether training is improving or where learning-rate-decay events happened.
Diagnosis	Single-step loss values are very noisy because each minibatch is a small random sample. Plotting raw values buries the underlying trend.
Action	Reshape the loss list into rows of about 1000 and take the mean along each row: <code>torch.tensor(losses).view(-1, 1000).mean(1)</code> . The smoothed curve makes trends visible.

Pitfall 6 · Forgetting to rerun a cell after edit

Symptom	Code looks correct in the notebook but produces wrong results. The cell you just edited contains the right code but the runtime still has the old version.
Diagnosis	Jupyter notebooks have stateful execution. Editing a cell does not automatically re-execute it. If you edit a model definition but forget to rerun the cell, the model object in memory is still the old version.
Action	Always rerun every cell you edit, and consider 'Restart and Run All' before any serious experiment. The bug Karpathy hit in Chapter 9 was exactly this: he edited the cell that creates the model but did not rerun it, so the model object was stale.

28. Summary and where the series goes next

Chapter overview

Time to consolidate. This chapter lists the most important takeaways from the lecture and previews what is coming next in the makemore series.

The eight things to take away

- **1. Crushing too much information at once wastes depth.** A flat MLP that takes a long context and immediately fuses all of it into a single hidden layer destroys temporal structure that later layers cannot recover.
- **2. Hierarchical fusion is the alternative.** Build the network so each layer fuses only pairs of adjacent positions. The receptive field doubles with each layer, growing exponentially while keeping per-layer fusion simple.
- **3. PyTorch's matmul broadcasts over leading dimensions.** A weight matrix of shape (in, out) can be applied to a tensor of any rank, as long as the last dimension matches in. This is what makes FlattenConsecutive + Linear so cheap.
- **4. Always print shapes when building a new architecture.** Shape bugs are silent and pernicious. The discipline of forwarding a small batch and printing every layer's output is the cheapest insurance you can buy.
- **5. BatchNorm has a separate code path for every input rank.** The 3D version of BatchNorm needs to reduce over both leading dimensions (0, 1), not just dim 0. The 2D version reduces over dim 0 alone. Mixing them up produces a layer that trains but is silently doing the wrong computation.
- **6. Always toggle training/eval mode before evaluating or sampling.** BatchNorm uses different code paths in the two modes. Forgetting to toggle is the most common bug in deep learning workflows.

- **7. Convolutions are an efficiency optimization, not a different model.** The hierarchical computation we built can be implemented with convolutions to share work across overlapping receptive fields. The model is the same; the kernels are different.
- **8. Develop in Jupyter, train in scripts.** Karpathy's recommended workflow, and the standard pattern in industry research labs. Notebooks for shape-checking and prototype iteration; scripts for serious training runs.

Where the series goes next

Karpathy outlines several directions for future lectures. The two most likely next steps:

- **Implement WaveNet with actual convolutions.** Keep the hierarchical topology but use dilated causal convolutions to implement it efficiently. This is the natural follow-up: same model, better mechanics.
- **Recurrent neural networks.** RNNs, LSTMs, GRUs — the family of architectures that processes sequences one position at a time, with hidden state carrying information forward. These were the dominant sequence models for years before Transformers.
- **Transformers.** The architecture that has replaced almost everything else in modern NLP. Attention-based, parallelizable, scalable. The natural endpoint of the makemore series.
- **Better experimental harness.** Karpathy is candid that we have been guessing and checking. A proper sweep over hyperparameters with a clean experimental setup would likely find more improvement in the current architecture alone.

The single thing to remember if you remember nothing else

Architecture matters. Until this lecture, every gain came from polishing the same MLP. This lecture showed what a structural change actually buys: by replacing the flat fusion with hierarchical fusion and then scaling up, we crossed the 2.0 validation loss barrier that stood for four lectures. The same will be true going forward: the move from MLPs to RNNs, from RNNs to Transformers, from scaled-up Transformers to scaled-up Transformers with better training — each will deliver bigger gains than any amount of tweaking within an existing architecture could.

29. Series-level recap: where we have been since Part 1

Chapter overview

Part 5 is a natural place to take stock of the whole makemore series. The five lectures covered five distinct topics, each building on the last, and together they constitute a compact tour of modern neural network practice. This chapter recaps what each lecture taught and what carries forward.

Part 1: The bigram counting model

We built a character-level language model based on counting pairs of consecutive characters in the training set. The model has no parameters in the deep learning sense — just a table of counts — but it establishes the framing: every word can be modelled as a sequence of conditional probabilities. The bigram model gave us a val loss of 2.45 and samples that were vaguely name-like but mostly gibberish.

Part 2: The multilayer perceptron

We replaced the count table with a learned model: a small MLP that takes three previous characters, embeds each into a 10-dimensional vector, concatenates them, and predicts the next character. Training with minibatch SGD and step learning rate decay brought the val loss to 2.17. We learned about embeddings, cross-entropy loss, the train/dev/test split, and the lr finder.

Part 3: Activations, gradients, BatchNorm

We diagnosed the Part 2 MLP and fixed three problems: confidently-wrong initialization, saturated tanh, and the more general problem of activation/gradient drift across layers. We introduced Kaiming initialization and Batch Normalization. We built the PyTorch-style module library and the diagnostic toolkit (activation histograms, gradient histograms, the update-to-data ratio). Final val loss: 2.10.

Part 4: Becoming a Backprop Ninja

We deleted `loss.backward()` and wrote the entire backward pass by hand, deriving every gradient through the MLP from scratch. We introduced the shape-matching shortcut for matrix-multiplication backward, the scatter-add pattern for indexing backward, and the analytical one-shot forms of cross-entropy and BatchNorm backward. The final loss matched Part 3's result, since we were doing the same computation with the same gradients — the value was in understanding, not improvement.

Part 5: Building a WaveNet

We outgrew the flat MLP and built a hierarchical, tree-like architecture inspired by WaveNet. The block size increased from 3 to 8, we added Embedding/FlattenConsecutive/Sequential modules to our framework, discovered the matmul broadcasting fact and the BatchNorm 3D bug, and scaled up to 76K parameters reaching the first sub-2.0 val loss in the series: 1.993. We finished with a preview of how convolutions would implement this architecture more efficiently.

What carries forward

Most of what we have built is universal. The diagnostic discipline from Part 3 (printing stats, watching update-to-data ratios) applies to every neural network you will ever train. The gradient-level intuition from Part 4 applies to every architecture. The PyTorch-style module abstraction from Parts 3-5 is exactly how real PyTorch code is structured. The hierarchical-fusion intuition from Part 5 generalizes to convolutional networks, Transformers, and anything else that processes structured input. Five lectures, an enormous fraction of the deep learning craft.

30. Glossary

This glossary defines every technical term used in the handbook in a single, scannable place. Look up anything you do not recognize here first.

Autoregressive model. A model that predicts the next element of a sequence as a function of the previous elements. Used in language modelling, audio generation, and other sequential prediction tasks. Our character-level model is autoregressive: it predicts character $t+1$ from characters 1 through t .

Batch dimension. Any leading dimension of a tensor whose elements are treated independently. In our hierarchical model the input has shape (B, T, C); both B and T are effectively batch dimensions as far as the Linear layer is concerned (the matmul broadcasts over both).

Block size. The length of the context window used for prediction. Part 2 used `block_size=3`; Part 5 increases it to 8.

Causal. A model whose prediction at position t depends only on positions 0 through $t-1$, never on future positions. Required for autoregressive generation, where you do not have access to future tokens at inference time. WaveNet uses “dilated causal convolutions” specifically because of this constraint.

Channels-first vs channels-last. Two conventions for ordering tensor dimensions in sequence models. Channels-first (NCL) puts the feature dimension before the time dimension; channels-last (NLC) puts it after. PyTorch's `nn.BatchNorm1d` uses channels-first; our convention is channels-last.

Convolution (1D). A linear operation that slides a small filter across the time dimension of an input sequence, applying the same weights at every position. Equivalent to a Linear layer with weight sharing across positions.

Dilated convolution. A convolution that skips over input positions in a controlled way, increasing the receptive field without increasing the filter size. Doubling the dilation at each layer produces exponential receptive field growth with linear depth.

Embedding module. A wrapper class around an embedding table. Provides the standard module API (constructor creates parameters, `__call__` does the forward, `parameters()` returns the trainable tensors). Equivalent to `torch.nn.Embedding`.

FlattenConsecutive(n). Karpathy's custom layer that reshapes a tensor of shape (B, T, C) into (B, T/n, C*n), packing every n consecutive positions into the last dimension. The reshape is purely a view, so it costs no memory or compute.

Gated activation unit. A non-linearity of the form $\tanh(x_a) * \text{sigmoid}(x_b)$, where x_a and x_b are separate linear projections. Used in WaveNet and the LSTM family. The sigmoid acts as a learned gate.

Hierarchical fusion. An architectural pattern where each layer of the network fuses only a small number of adjacent positions from the previous layer. After $\log(\text{context})$ layers, every output position has seen the

entire context window. Used by WaveNet, tree LSTMs, and several other models.

Kaiming initialization. Drawing weights from a distribution with standard deviation $\text{gain} / \sqrt{\text{fan_in}}$. Preserves the standard deviation of activations across layers. Default in our Linear class.

Last-dimension matmul. PyTorch's `x @ w` performs matrix multiplication only on the last dimension of `x` with `w`. Leading dimensions are treated as batch. This is what lets a single Linear layer apply to data with any number of leading dimensions.

Layer module. A class that encapsulates one layer of a neural network. Has a constructor that creates parameters, a `__call__` method that does the forward pass, and a `parameters()` method that returns the trainable tensors. Mimics PyTorch's `nn.Module` API.

Receptive field. The set of input positions that influence a given output position. In our hierarchical model, after three stages each output position has a receptive field of 8 input positions.

Residual connection. Adding a layer's input to its output, so the layer learns a residual transformation rather than the full transformation. Lets gradients flow more easily through deep stacks. Not implemented in our lecture but central to modern deep architectures.

Running statistics. Exponential moving averages of the batch mean and variance that BatchNorm maintains during training, then uses at inference time when batch statistics are unavailable. Updated each forward pass with a configurable momentum.

Sequential container. A class that wraps a list of layers and runs them in order during the forward pass. Equivalent to `torch.nn.Sequential`. Lets the whole model be defined as a single object with a clean `.parameters()` and `__call__()`.

Skip connection. A connection that bypasses one or more layers and contributes to a later layer directly. WaveNet collects intermediate outputs via skip connections to produce the final prediction.

Squeeze. Remove a singleton (size-1) dimension from a tensor. Used inside `FlattenConsecutive` when the time dimension collapses to 1 after the final fusion, so the result is a 2D tensor that feeds cleanly into the final Linear.

Tree-like topology. The shape of the computation in a hierarchical fusion model: leaves are individual input positions, internal nodes fuse pairs of children, the root combines the entire context. Our three-stage WaveNet is a binary tree with eight leaves.

Train mode vs eval mode. Two modes that stateful layers (BatchNorm, Dropout) operate in. Train mode uses fresh statistics from the current batch and updates internal state. Eval mode uses fixed running statistics and does not update state. Forgetting to toggle is the most common bug in deep learning.

Update-to-data ratio. The standard deviation of $\text{lr} * \text{grad}$ divided by the standard deviation of data, for a given parameter. Healthy value $\sim 1e-3$ on a log scale. Introduced in Part 3.

View (PyTorch). A reshape operation that does not copy data — it just creates a new tensor that interprets the same memory with a different shape. $O(1)$ regardless of tensor size.

WaveNet. A 2016 DeepMind paper introducing an autoregressive model for raw audio generation. Uses dilated causal convolutions in a hierarchical, tree-like topology. Our lecture borrows the topology and skips the convolution mechanics.

End of Part 5. The makemore series so far has taken us from a counting bigram model to a 76K-parameter hierarchical network that produces noticeably name-like samples and crosses below 2.0 validation loss for the first time. The architectural toolkit, diagnostic discipline, gradient-level intuition, and module abstraction we have built apply directly to the next stages of the journey: convolutional implementations of this same model, recurrent networks, and ultimately Transformers.